Advanced topics

BR2_EXTERNAL: principle

- Storing your custom packages, custom configuration files and custom *defconfigs* inside the Buildroot tree may not be the most practical solution
  - Doesn't cleanly separate open-source parts from proprietary parts
  - Makes it harder to upgrade Buildroot
- The BR2_EXTERNAL mechanism allows to store your own package recipes, *defconfigs* and other artefacts **outside** of the Buildroot source tree.
- It is possible to use several BR2_EXTERNAL trees, to further separate various aspects of your project.
- Note: can only be used to add new packages, not to override existing Buildroot packages

BR2_EXTERNAL: example organization

- project/
  - buildroot/
    - The Buildroot source code, cloned from Git, or extracted from a release tarball.
  - external1/
  - external2/
    - Two external trees
  - output-build1/
  - output-build2/
    - Several *output* directories, to build various configurations
  - custom-app/
  - custom-lib/
    - The source code of your custom applications and libraries.

Using BR2_EXTERNAL

- Specify, as a colon-separated list, the *external* directories in BR2_EXTERNAL
- Not a configuration option, only an **environment variable** to be passed on the command line

```
make BR2_EXTERNAL=/path/to/external1:/path/to/external2
```

- **Automatically saved** in the hidden .br2-external.mk file in the output directory
  - no need to pass BR2_EXTERNAL at every make invocation
  - can be changed at any time by passing a new value, and removed by passing an empty value
- Can be either an **absolute** or a **relative** path, but if relative, important to remember that it's relative to the Buildroot source directory

BR2_EXTERNAL: important files

- Each *external* directory must contain:
  - `external.desc`, which provides a name and description
  - `Config.in`, configuration options that will be included in *menuconfig*
  - `external.mk`, will be included in the make logic
- If `configs` exists, it will be used when listing all *defconfigs*

BR2_EXTERNAL: recommended structure

```
+-- board/
|   +-- <company>/
|       +-- <boardname>/
|           +-- linux.config
|           +-- busybox.config
|           +-- <other configuration
files>
|           +-- post_build.sh
|           +-- post_image.sh
|           +-- rootfs_overlay/
|           |   +-- etc/
|           |   +-- <some file>
|           +-- patches/
|               +-- libbar/
|                   +-- <some patches>
|
+-- configs/
|   +-- <boardname>_defconfig
|
```

```
+-- package/
|   +-- <company>/
|       +-- package1/
|       |    +-- Config.in
|       |    +-- package1.mk
|       +-- package2/
|           +-- Config.in
|           +-- package2.mk
|
+-- Config.in
+-- external.mk
+-- external.desc
```

BR2_EXTERNAL: external.desc

- File giving metadata about the *external tree*
- Mandatory `name` field, using characters in the set `[A-Za-z0-9_]`. Will be used to define `BR2_EXTERNAL_<NAME>_PATH` available in `Config.in` and `.mk` files, pointing to the external tree directory.
- Optional `desc` field, giving a free-form description of the external tree. Should be reasonably short.
- Example

```
name: FOOBAR
desc: Foobar Company
```

BR2_EXTERNAL: main Config.in

- Custom configuration options
- Configuration options for the external packages
- The \$BR2_EXTERNAL_<NAME>_PATH variable is available, where NAME is defined in external.desc

Example Config.in

```
source "\$BR2_EXTERNAL_<NAME>_PATH/package/package1/Config.in"
source "\$BR2_EXTERNAL_<NAME>_PATH/package/package2/Config.in"
```

BR2_EXTERNAL: external.mk

- Can include custom *make* logic
- Generally only used to include the package `.mk` files

Example `external.mk`

```
make
include $(sort $(wildcard $(BR2_EXTERNAL_<NAME>_PATH)/package/*/*.mk))
```

Use BR2_EXTERNAL in your configuration

- In your Buildroot configuration, don't use absolute paths for the *rootfs overlay*, the *post-build scripts*, *global patch directories*, etc.
- If they are located in an external tree, you can use `$(BR2_EXTERNAL_<NAME>_PATH)` in your Buildroot configuration options.
- With the recommended structure shown before, a Buildroot configuration would look like:

```
BR2_GLOBAL_PATCH_DIR="$(BR2_EXTERNAL_<NAME>_PATH)/board/<company>/<boardname>/patches/"
...
BR2_ROOTFS_OVERLAY="$(BR2_EXTERNAL_<NAME>_PATH)/board/<company>/<boardname>/rootfs_overlay/"
...
BR2_ROOTFS_POST_BUILD_SCRIPT="$(BR2_EXTERNAL_<NAME>_PATH)/board/<company>/<boardname>/post_build.sh"
BR2_ROOTFS_POST_IMAGE_SCRIPT="$(BR2_EXTERNAL_<NAME>_PATH)/board/<company>/<boardname>/post_image.sh"
...
BR2_LINUX_KERNEL_USE_CUSTOM_CONFIG=y BR2_LINUX_KERNEL_CUSTOM_CONFIG_FILE="$(BR2_EXTERNAL_<NAME>_PATH)/board/
<company>/<boardname>/linux.config"
```

Examples of BR2_EXTERNAL trees

- There are a number of publicly available BR2_EXTERNAL trees, especially from hardware vendors:
  - buildroot-external-st, maintained by Bootlin in partnership with ST, containing example configurations for the STM32MP1 platforms. https://github.com/bootlin/buildroot-external-st
  - buildroot-external-microchip, containing example configurations, additional packages and demo applications for Microchip ARM platforms. https://github.com/linux4sam/buildroot-external-microchip
  - buildroot-external-boundary, containing example configurations for Boundary Devices boards, mainly based on NXP i.MX processors. https://github.com/boundarydevices/buildroot-external-boundary

Package-specific targets: basics

- Internally, each package is implemented through a number of package-specific *make targets*
  - They can sometimes be useful to call directly, in certain situations.
- The targets used in the normal build flow of a package are:
  - `<pkg>`, fully build and install the package
  - `<pkg>-source`, just download the source code
  - `<pkg>-extract`, download and extract
  - `<pkg>-patch`, download, extract and patch
  - `<pkg>-configure`, download, extract, patch and configure
  - `<pkg>-build`, download, extract, patch, configure and build
  - `<pkg>-install-staging`, download, extract, patch, configure and do the staging installation (target packages only)
  - `<pkg>-install-target`, download, extract, patch, configure and do the target installation (target packages only)
  - `<pkg>-install`, download, extract, patch, configure and install

Package-specific targets: example (1)

```
$ make strace
>>> strace 4.10 Extracting
>>> strace 4.10 Patching
>>> strace 4.10 Updating config.sub and config.guess
>>> strace 4.10 Patching libtool
>>> strace 4.10 Configuring
>>> strace 4.10 Building
>>> strace 4.10 Installing to target
$ make strace-build
... nothing ...
$ make ltrace-patch
>>> ltrace 0896ce554f80afdcba81d9754f6104f863dea803 Extracting
>>> ltrace 0896ce554f80afdcba81d9754f6104f863dea803 Patching
$ make ltrace
>>> argp-standalone 1.3 Extracting
>>> argp-standalone 1.3 Patching
>>> argp-standalone 1.3 Updating config.sub and config.guess
>>> argp-standalone 1.3 Patching libtool
[...]
>>> ltrace 0896ce554f80afdcba81d9754f6104f863dea803 Configuring
>>> ltrace 0896ce554f80afdcba81d9754f6104f863dea803 Autoreconfiguring
>>> ltrace 0896ce554f80afdcba81d9754f6104f863dea803 Patching libtool
>>> ltrace 0896ce554f80afdcba81d9754f6104f863dea803 Building
>>> ltrace 0896ce554f80afdcba81d9754f6104f863dea803 Installing to target
```

Package-specific targets: advanced

- Additional useful targets
  - `make <pkg>-show-depends`, show the package dependencies
  - `make <pkg>-graph-depends`, generates a dependency graph
  - `make <pkg>-dirclean`, completely remove the package source code directory. The next `make` invocation will fully rebuild this package.
  - `make <pkg>-reinstall`, force to re-execute the installation step of the package
  - `make <pkg>-rebuild`, force to re-execute the build and installation steps of the package
  - `make <pkg>-reconfigure`, force to re-execute the configure, build and installation steps of the package.

Package-specific targets: example (2)

```
$ make strace
>>> strace 4.10 Extracting
>>> strace 4.10 Patching
>>> strace 4.10 Updating config.sub and config.guess
>>> strace 4.10 Patching libtool
>>> strace 4.10 Configuring
>>> strace 4.10 Building
>>> strace 4.10 Installing to target
$ ls output/build/
strace-4.10 [...]
$ make strace-dirclean rm -Rf /home/thomas/projets/buildroot/output/build/strace-4.10
$ ls output/build/
[... no strace-4.10 directory ...]
```

Package-specific targets: example (3)

```
$ make strace
>>> strace 4.10 Extracting
>>> strace 4.10 Patching
>>> strace 4.10 Updating config.sub and config.guess
>>> strace 4.10 Patching libtool
>>> strace 4.10 Configuring
>>> strace 4.10 Building
>>> strace 4.10 Installing to target
$ make strace-rebuild
>>> strace 4.10 Building
>>> strace 4.10 Installing to target
$ make strace-reconfigure
>>> strace 4.10 Configuring
>>> strace 4.10 Building
>>> strace 4.10 Installing to target
```

make show-info

- `make show-info` outputs JSON text that describes the current configuration: enabled packages, in which version, their license, tarball, dependencies, etc.
- Can be useful for post-processing, build analysis, license compliance, etc.

```
$ make show-info | jq .
{
  "busybox": {
    "type": "target",
    "virtual": false,
    "version": "1.31.1",
    "licenses": "GPL-2.0",
    "dl_dir": "busybox",
    "install_target": true,
    "install_staging": false,
    "install_images": false,
    "downloads": [
      {
        "source": "busybox-1.31.1.tar.bz2",
        "uris": [
          "http+http://www.busybox.net/downloads",
          "http|urlencode+http://sources.buildroot.net/busybox",
        ]
      }
    ],
    "dependencies": [
      "host-skeleton",
      "host-tar",
      "skeleton",
      "toolchain"
    ],
    "reverse_dependencies": []
  },
```

Understanding rebuilds (1)

- Doing a **full rebuild** is achieved using:

```
$ make clean all
```

  - It will completely remove all build artefacts and restart the build from scratch
- Buildroot **does not try to be smart**
  - once the system has been built, if a configuration change is made, the next `make` will **not apply all the changes** made to the configuration.
  - being smart is very, very complicated if you want to do it in a reliable way.

Understanding rebuilds (2)

- When a package has been built by Buildroot, Buildroot keeps a **hidden file** telling that the package has been built.
  - Buildroot will therefore *never* rebuild that package, unless a **full rebuild is done**, or this specific package is **explicitly rebuilt**.
  - Buildroot does not *recurse* into each package at each `make` invocation, it would be too time-consuming. So if you change one source file in a package, Buildroot does not know it.
- When `make` is invoked, Buildroot **will always**:
  - Build the packages that have not been built in a previous build and install them to the target
  - Cleanup the target root filesystem from useless files
  - Run *post-build* scripts, copy *rootfs overlays*
  - Generate the root filesystem images
  - Run *post-image* scripts

Understanding rebuilds: scenarios (1)

- If you enable a new package in the configuration, and run `make`
  - Buildroot will build it and install it
  - However, other packages that may benefit from this package will not be rebuilt automatically
- If you remove a package from the configuration, and run `make`
  - Nothing happens. The files installed by this package are not removed from the target filesystem.
  - Buildroot does not track which files are installed by which package
  - Need to do a full rebuild to get the new result. Advice: do it only when really needed.
- If you change the sub-options of a package that has already been built, and run `make`
  - Nothing happens.
  - You can force Buildroot to rebuild this package using `make <pkg>-reconfigure` or `make <pkg>-rebuild`.

Understanding rebuilds: scenarios (2)

- If you make a change to a *post-build* script, a *rootfs overlay* or a *post-image* script, and run `make`
  - This is sufficient, since these parts are re-executed at every `make` invocation.
- If you change a fundamental system configuration option: architecture, type of toolchain or toolchain configuration, init system, etc.
  - You **must do a full rebuild**
- If you change some source code in `output/build/<foo>-<version>/` and issue `make`
  - The package will not be rebuilt automatically: Buildroot has a *hidden file* saying that the package was already built.
  - Use `make <pkg>-reconfigure` or `make <pkg>-rebuild`
  - And remember that doing changes in `output/build/<foo>-<version>/` can only be temporary: this directory is removed during a `make clean`.

Tips for building faster

- Build time is often an issue, so here are some tips to help
  - Use fast hardware: lots of RAM, and SSD
  - Do not use virtual machines
  - You can enable the `ccache` *compiler cache* using `BR2_CCACHE`
  - Use external toolchains instead of internal toolchains
  - Learn about rebuilding only the few packages you actually care about
  - Build everything locally, do not use NFS for building
  - Remember that you can do several independent builds in parallel in different output directories

Support for top-level parallel build (1)

- Buildroot normally builds packages **sequentially**, one after the other.
- Calling Buildroot with `make -jX` has no effect
- Parallel build is used *within* the build of each package: Buildroot invokes each package build system with `make -jX`
  - This level of parallelization is controlled by `BR2_JLEVEL`
  - Defaults to 0, which means Buildroot auto-detects the number of CPUs cores
- Buildroot 2020.02 has introduced **experimental** support for top-level parallel build
  - Allows to build multiple different packages in parallel
  - Of course taking into account their dependencies
  - Allows to better use multi-core machines
  - Reduces build time significantly

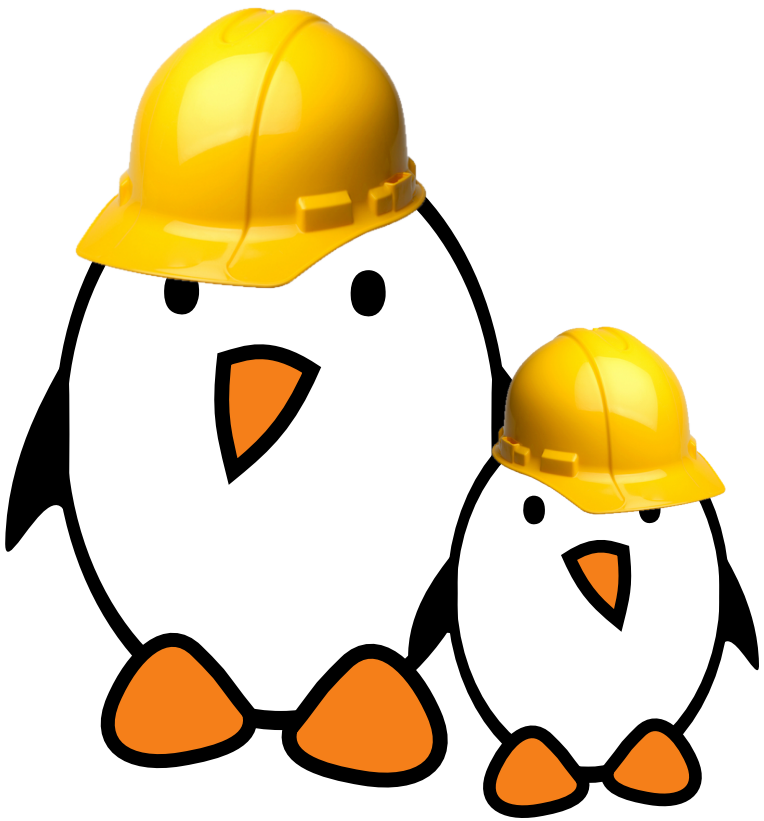
Support for top-level parallel build (2)

- To use this experimental support:
  1. Enable `BR2_PER_PACKAGE_DIRECTORIES=y`
  2. Build with `make -jX`
- The *per-package* option ensures that each package uses its own `HOST_DIR`, `STAGING_DIR` and `TARGET_DIR` so that different packages can be built in parallel with no interference
- See `$(O)/per-package/<pkg>/`
- Limitations
  - Not yet supported by all packages, e.g *Qt5*
  - Absolutely requires that packages do not overwrite/change files installed by other packages
  - `<pkg>-reconfigure`, `<pkg>-rebuild`, `<pkg>-reinstall` not working

Reproducible builds

- Buildroot guarantees that for a given version/configuration, it will **always build the same components**, in the same version, with the same configuration.
- However, a number of aspects (time, user, build location) can affect the build and make two consecutive builds of the same configuration **not strictly identical**.
- `BR2_REPRODUCIBLE` enables experimental support for build reproducibility
- Goal: have **bit-identical results** when
  - Date/time is different (i.e. same build later)
  - Build location has the same path length

Practical lab - Advanced aspects



- Use `legal-info` for legal information extraction
- Use `graph-depends` for dependency graphing
- Use `graph-build` for build time graphing
- Use `BR2_EXTERNAL` to isolate the project-specific changes (packages, configs, etc.)